# Mini-Project 5: Sensor Dashboard GUI

## Goals

In this mini-project, you will develop a graphical user interface (GUI - pronounced like “gooy”) for your SensorDataAnalyzer class developed in Mini-Project 4. As engineers, we often have to develop tools that can be used by people without a technical background. GUI programming is an important skill that will allow you to develop simple, intuitive interfaces to your code, thereby making it more accessible to the general public. To create our GUI, we will use the MATLAB AppDesigner tool — itself a GUI — that makes layout and design particularly straightforward. Concepts covered from the lecture include:

- Object oriented programming
- Event driven programming and callbacks

An example of what your GUI will end up looking like is provided in Figure 1 below.

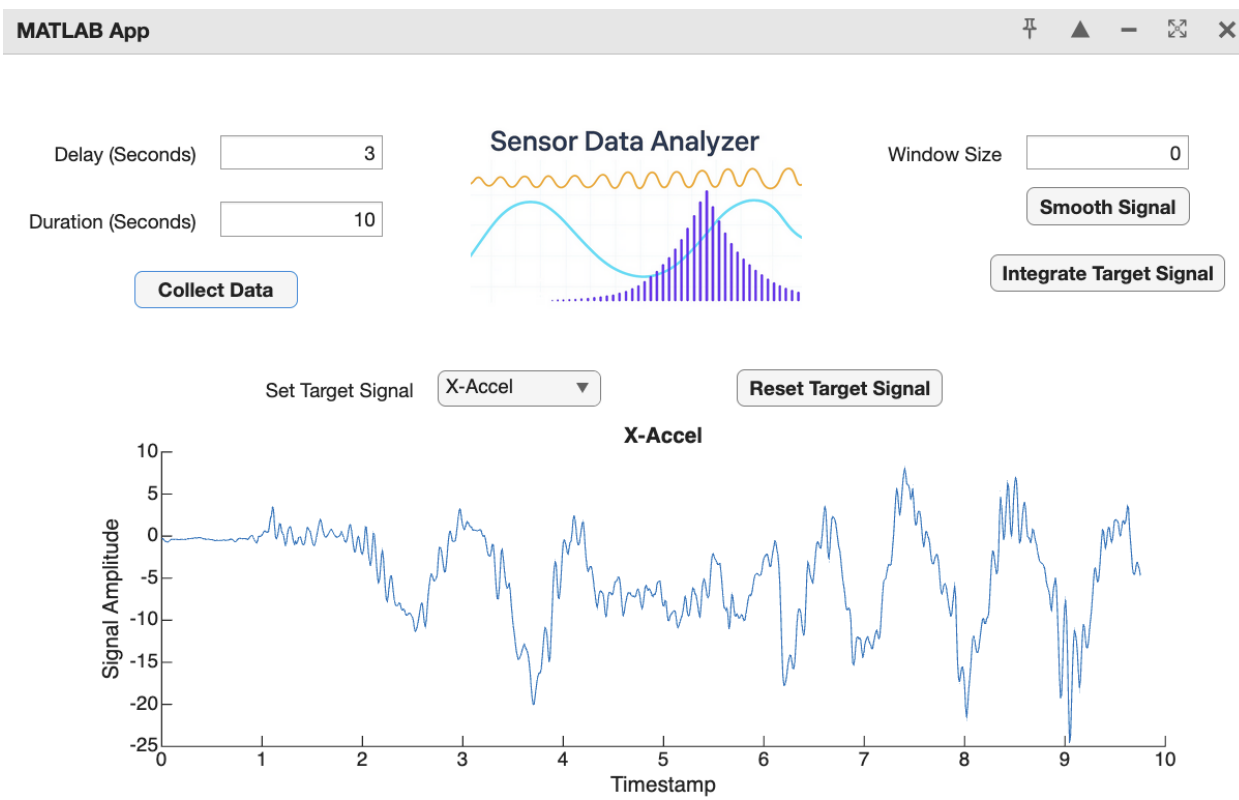


**Figure 1**: An example of what your final app should look like.

# Steps

## Step 1: Layout GUI Components

In this step, we will add the basic GUI components to our app. In subsequent steps, we will add *callbacks* that run dedicated pieces of code when a user clicks on a button or interacts with our app in some other way.

First, open the AppDesigner by clicking on the “Design App” icon as shown in Figure 2 below.

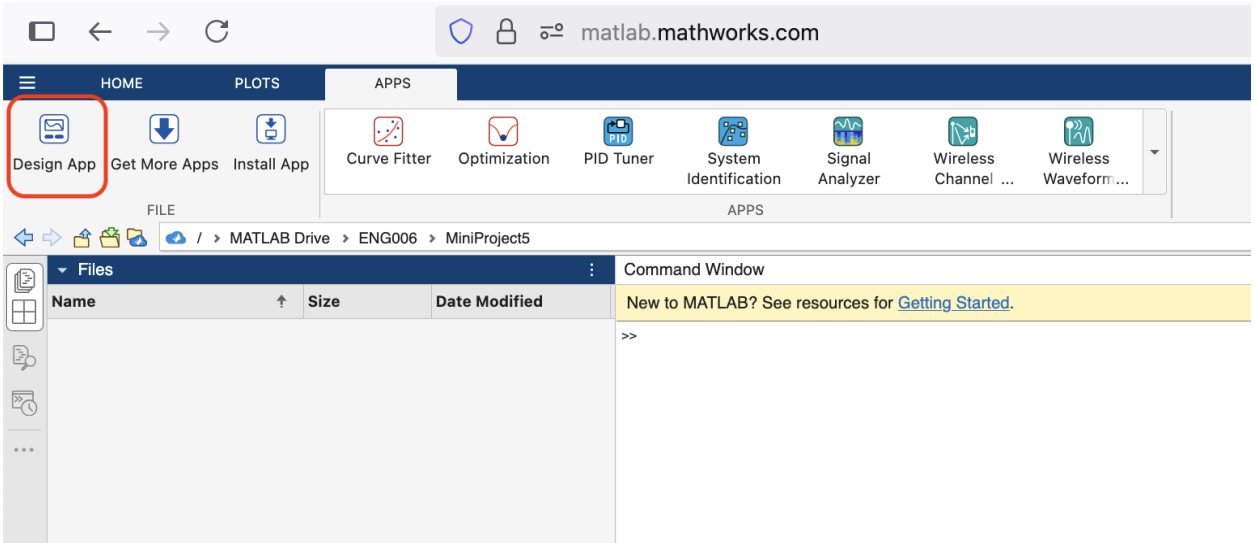


**Figure 2:** Opening the app designer.

Next, drag and drop components onto the blank canvas to create an app canvas that looks generally like the one shown in Figure 3 below. Your app doesn’t need to look completely identical, but it should have all the same components and be laid out in a generally reasonable way. Apps that are excessively cluttered or poorly laid out will lose points.

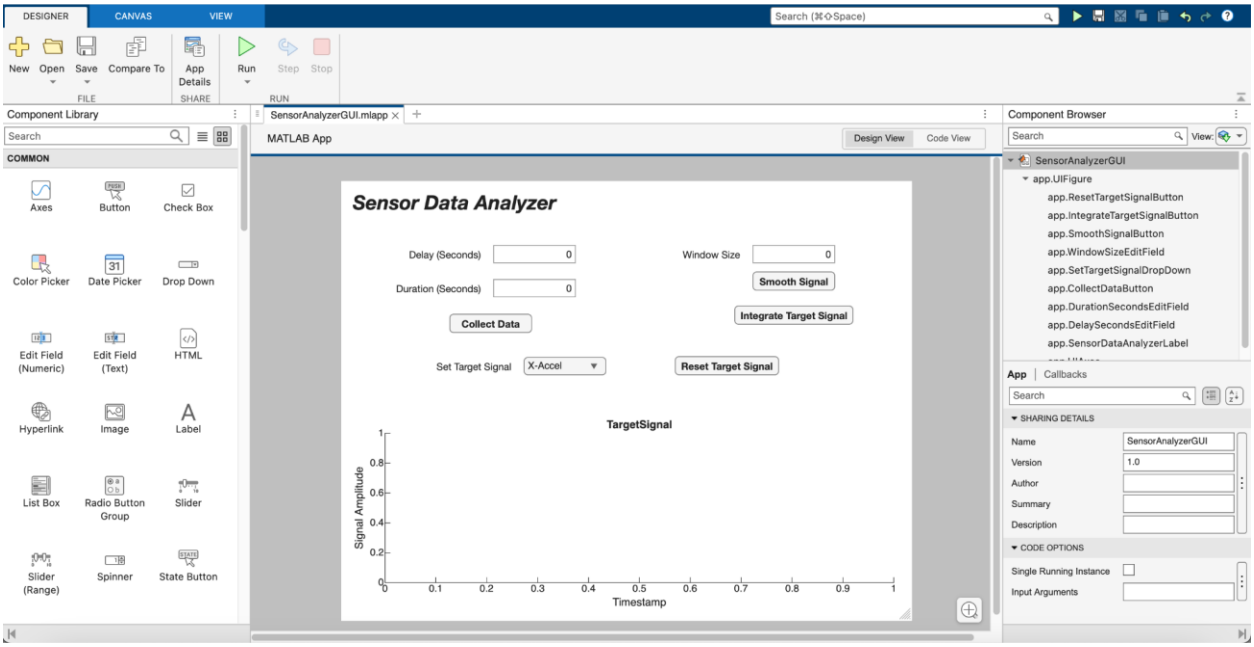


**Figure 3**: An example app canvas populated with interactive elements. Your canvas doesn't need to look the same, but it must contain all of the buttons/elements shown here, laid out in a reasonably intuitive way.

**Hints**:

1. You will need (at least) the following components:
    a. "Button," "Drop Down," "Edit Field (Numeric)," "Axes"
2. Many visual properties of a GUI component can be edited by clicking on the component and using the edit menu on the right-hand side of the screen. An example showing how to edit the text displayed on a button is shown in Figure 4 below.

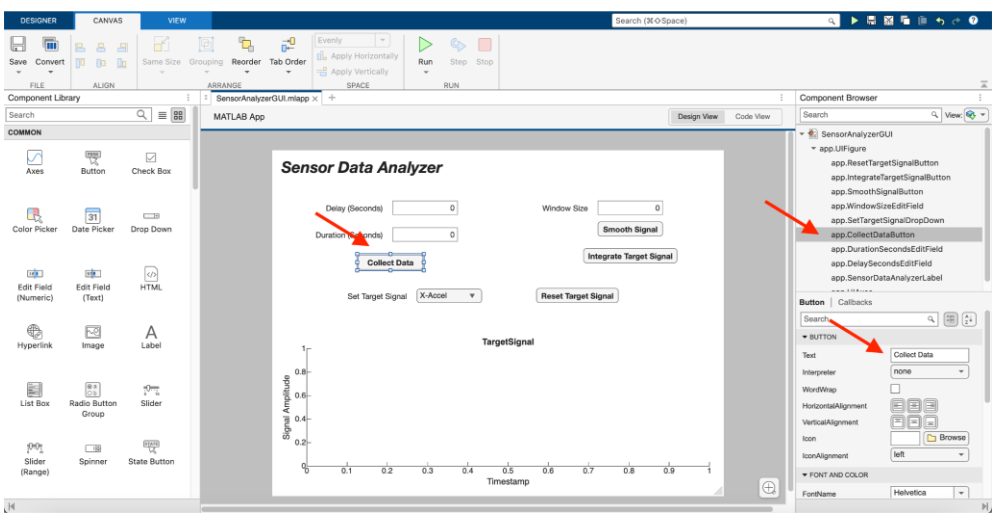


**Figure 4**: Editing properties of a GUI component using the edit menu. Click on the component to be edited and inspect the possible editable attributes in the menu to the right.

# Step 2: Add Startup Function and Custom Properties

A startup function can be used to perform certain actions when your app starts up for the first time. Our startup function needs to create an instance of your SensorDataAnalyzer class (from Mini-Project 4) and assign this to a local property of the app.

First, switch to the “code view” by clicking on the toggle shown in Figure 5 below. Code view will allow you to write custom code for your app.

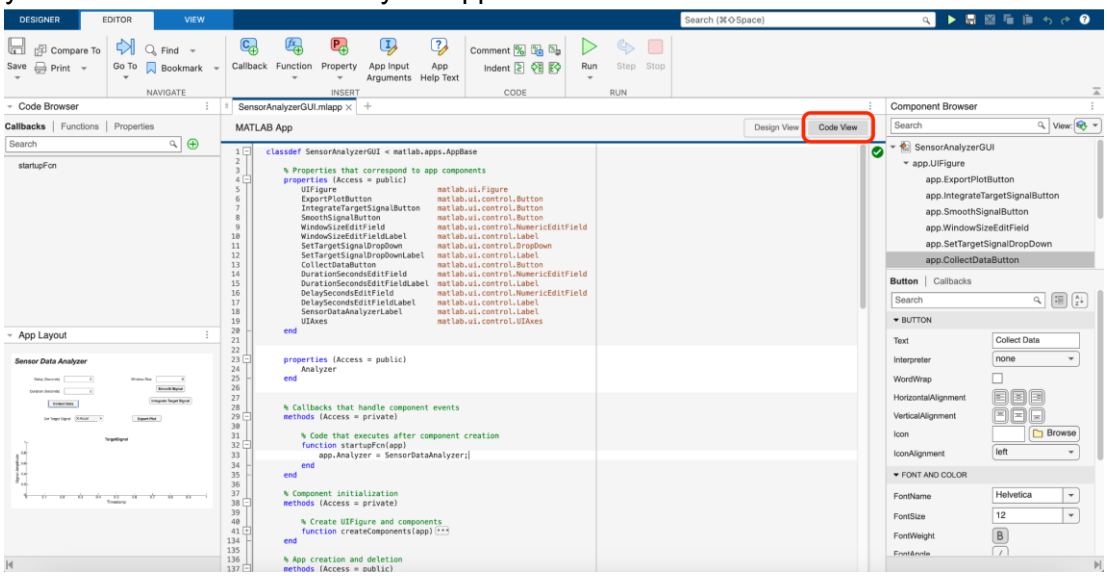


**Figure 5**: Opening “code view” in App Builder.

Next, we’ll add a “property” to store our instance of SensorDataAnalyzer. Click on the “Property” icon. You should now see an editable “properties” field. Add a property called “Analyzer” as shown in Figure 6 below:

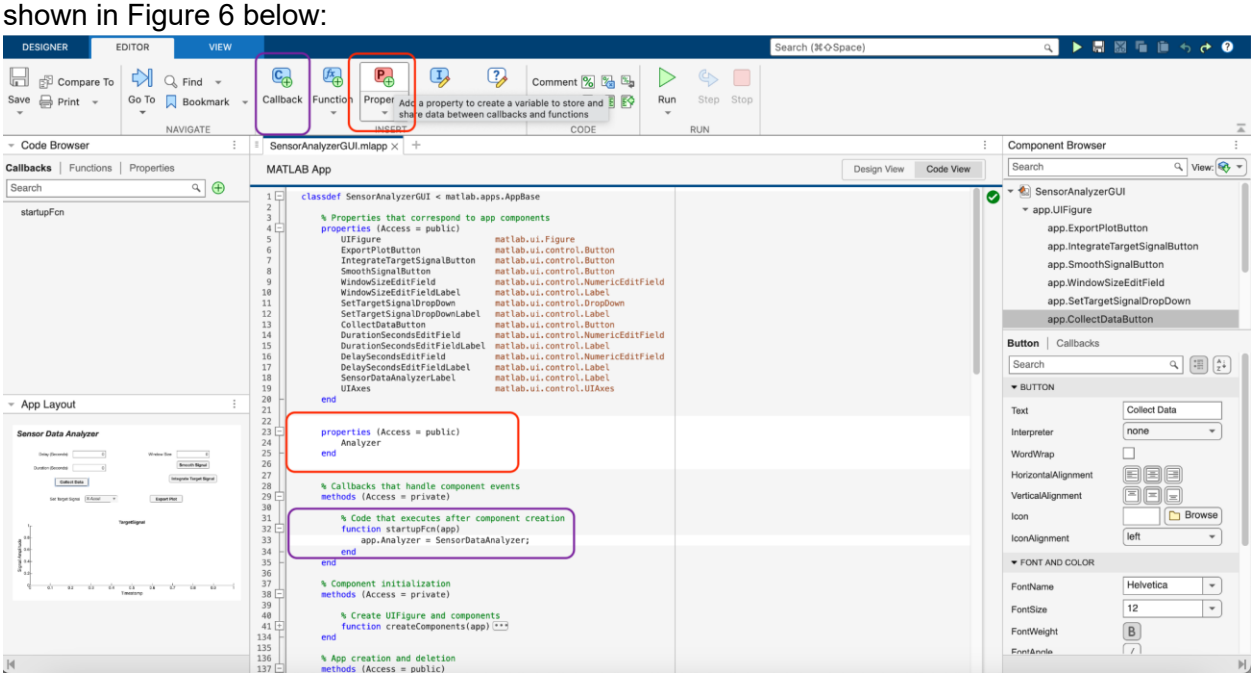

**Figure 6:** Adding a property and startup function. Steps for adding a property are highlighted in red. Steps for adding a startup function are highlighted in purple.

Next, we'll add a startup function to create a new instance of your SensorDataAnalyzer class when the app starts up. Click on "Callback" (highlighted in purple in Figure 6). Choose the option called "*Add Startup Fcn Callback"* (or something similar). You should now see an editable method called "startupFcn" as shown in purple in Figure 6 above. Create an instance of your SensorDataAnalyzer class (**hint**: the class file needs to be in the same folder as your app file) and assign it to the Analyzer property of the app.

# Step 3: Add Callbacks

In this step, we will add *callback* functions that can be used to perform some action when a component on your app is clicked/modified. We'll walk through the process for adding the callback for the "Collect Data" button in detail. You'll need to figure out the process for the others on your own : )

## Step 3.1: Add Collect Data Callback

First, switch back to "Design View" and right-click on the "Collect Data" button. In the menu that pops up, select "*Callbacks* > *Add ButtonPushedFcn Callback"* as shown in Figure 7 below.

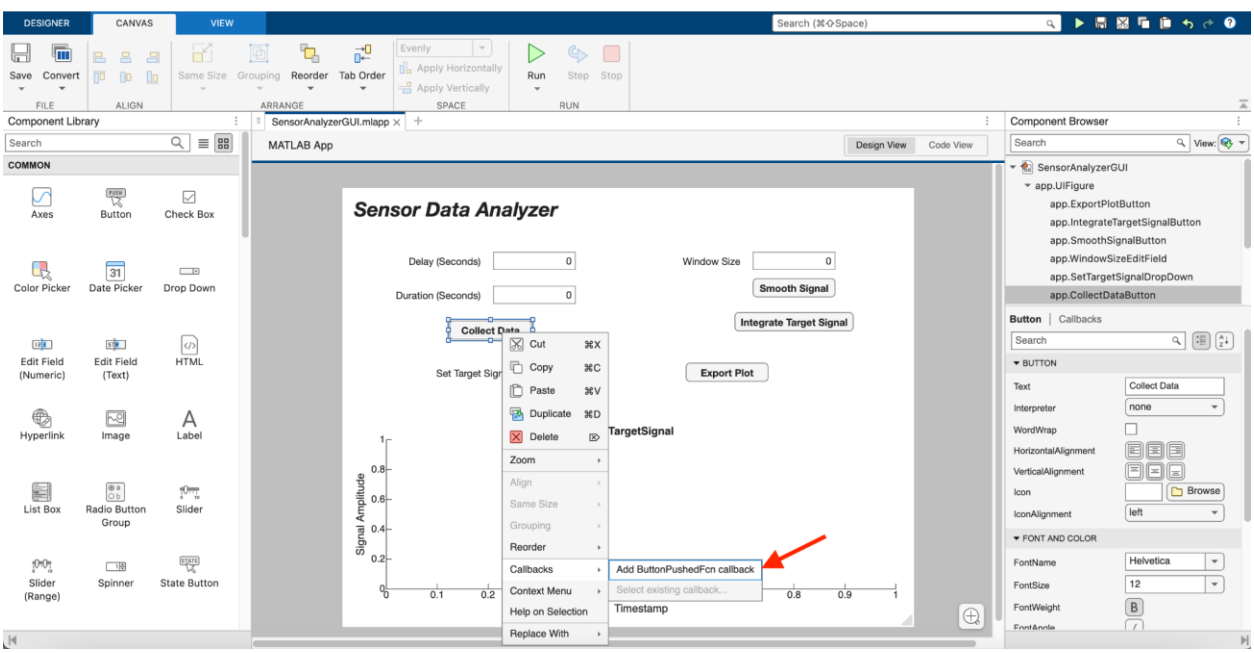

**Figure 7:** Adding a callback to be run when the user clicks on a button.

You should be taken back to the code view, where you'll see a blank function called something like "*CollectDataButtonPushed.*" When the user clicks on the "Collect Data" button, your app will

run this code. Fill in the function so it calls the “logSensorData” method of your “SensorDataAnalyzer” class with the delay and duration values entered by the user in the corresponding ‘edit-fields’ of your app. An example is shown in Figure 8 below.

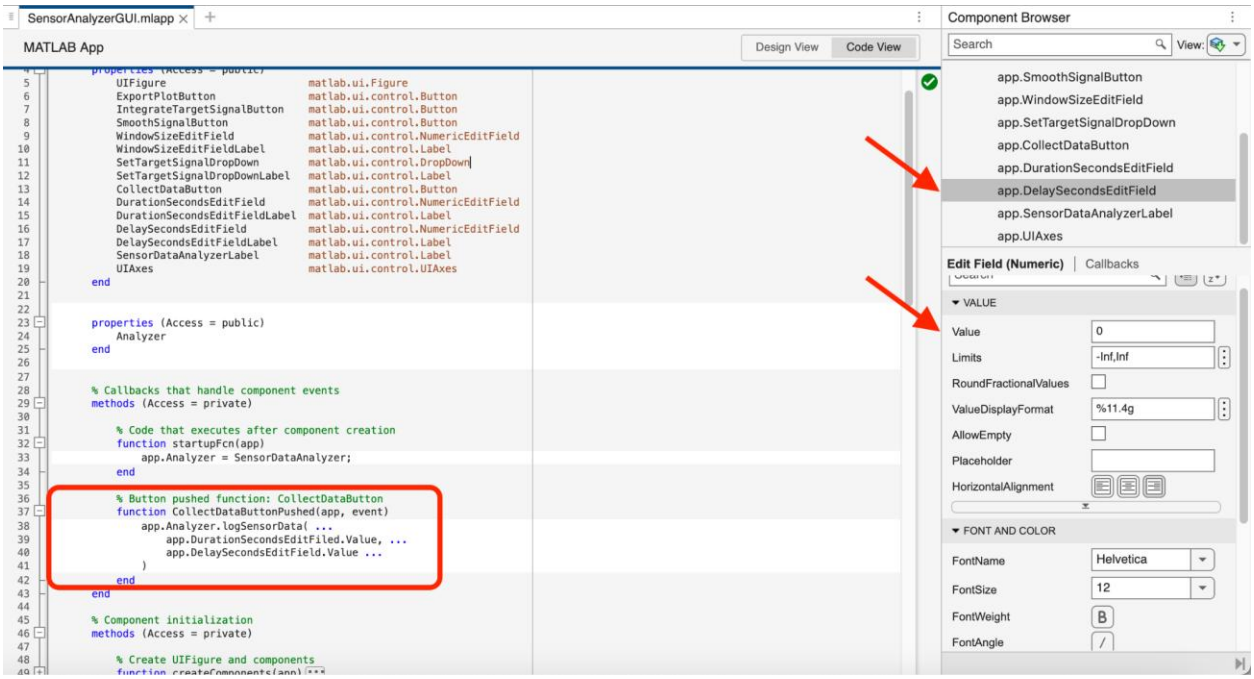


**Figure 8**: Adding a callback function to handle a button pushed event. Red arrows indicate accessible attributes of the “app.DelaySecondsEditField” property.

**Hint**: when the user enters a number in the edit field, this value is stored in a property called “Value” of the corresponding object in your app. To access this value in your code, you can do something like (exact syntax varies depending on what you called your edit field): `app.DelaySecondsEditField.Value` as shown in Figure 8 above. In general, you can get information about what properties are accessible for an object by clicking on it in the “Component Browser” and inspecting the properties list below as shown with red arrows in Figure 8. You can also right-click on the component in “Design View” and click on “Help on Selection” to get to a documentation page with information about the component.

When data collection has finished, set the TargetSignal to the value selected by the user from the dropdown menu you added in Step 1. This value is stored as the “Value” property of the corresponding app component property. In the example above it would be accessed by `app.SetTargetSignalDropDown.Value`.

Finally, plot the TargetSignal on the axes you added to the Canvas in Step 1 and title the plot with the TargetSignal selected by the user.

**Hint**: Our app creates an object called “UIAxes” to manage the plot on our app Canvas. We need to explicitly tell the “plotTargetSignal” method of our SensorDataAnalyzer class to plot on these axes rather than the default axes. The cleanest way to do this is by modifying your “plotTargetSignal” method so that it now has the signature:

**plotTargetSignal(obj, axes, titleStr)**

Where “axes” is a handle to the axes on which the plot should be drawn. You can pass an axis handle to the “plot” function as the first argument:

```
plot(axes, x, y, ___)
```

and do likewise for the “title” function:

```
title(axes, ___)
```

Modify your implementation of the **plotTargetSignal** method to plot on the app UIAxes. An example of how to call your method from the callback and set the title is as follows:

```
signalName = app.SetTargetSignalDropDown.Value;
app.Analyzer.setTargetSignal(signalName);
app.Analyzer.plotTargetSignal(app.UIAxes, signalName);
```

## Step 3.2: Test Your App

Run your app by pressing the green “play” button on the app developer. Your app should launch, and you should be able to collect data and plot it. You are strongly encouraged to debug your app *incrementally*. After adding each new callback below, debug your app to make sure the callback works as intended. This will be much easier than debugging everything all at once.

## Step 3.3: Add Callbacks for Other Components

Now that you’ve seen the process in Step 3.1, add the following callbacks:

1. **Smoothing** – When the “Smooth” button is pressed, call the *smooth* method from your SensorDataAnalyzer object. **You may use the smooth method implemented in the template provided on Canvas for mini-project 4**.
2. **Integration** – When the “Integrate” button is pressed, call the *integrate* method from your SensorDataAnalyzer object.
3. **Change Target Signal** – When the user selects a different signal from the “Set Target Signal” drop-down, update the current target signal accordingly. To do this, you can create a “ValueChangedFcn” callback from the pull-out menu from right-clicking on the dropdown.
4. **Reset Target Signal** – When the “Reset Target Signal” button is pressed, restore the original, unprocessed version of the target signal.

**Important:** After each callback completes, the updated signal should be displayed on the plot axes in your app (the same plot you used for showing the target signal earlier).

You should follow the same general procedure from Step 3.1 for creating these callbacks — right-click on the component in **Design View**, add the appropriate callback (e.g. ButtonPushed or ValueChanged), and then fill in the code to perform the intended action and re-plot the signal.

## Step 4: Add a Logo

Finally, let's add a background "logo" image to improve the look of our UI. An example is given in Figure 8 below. Create a logo image of your own choosing using photo-editing software or using an AI image generation tool (the image below was created using OpenAI Sora). Save the result as a PNG and place it in the folder where your app file is located. There are no specific requirements about what your logo must look like, but it needs to be unique and it must be non-trivial.

Add an "image" component to your app canvas from the "Design View" menu and choose your logo by clicking on "browse" and selecting your png file.

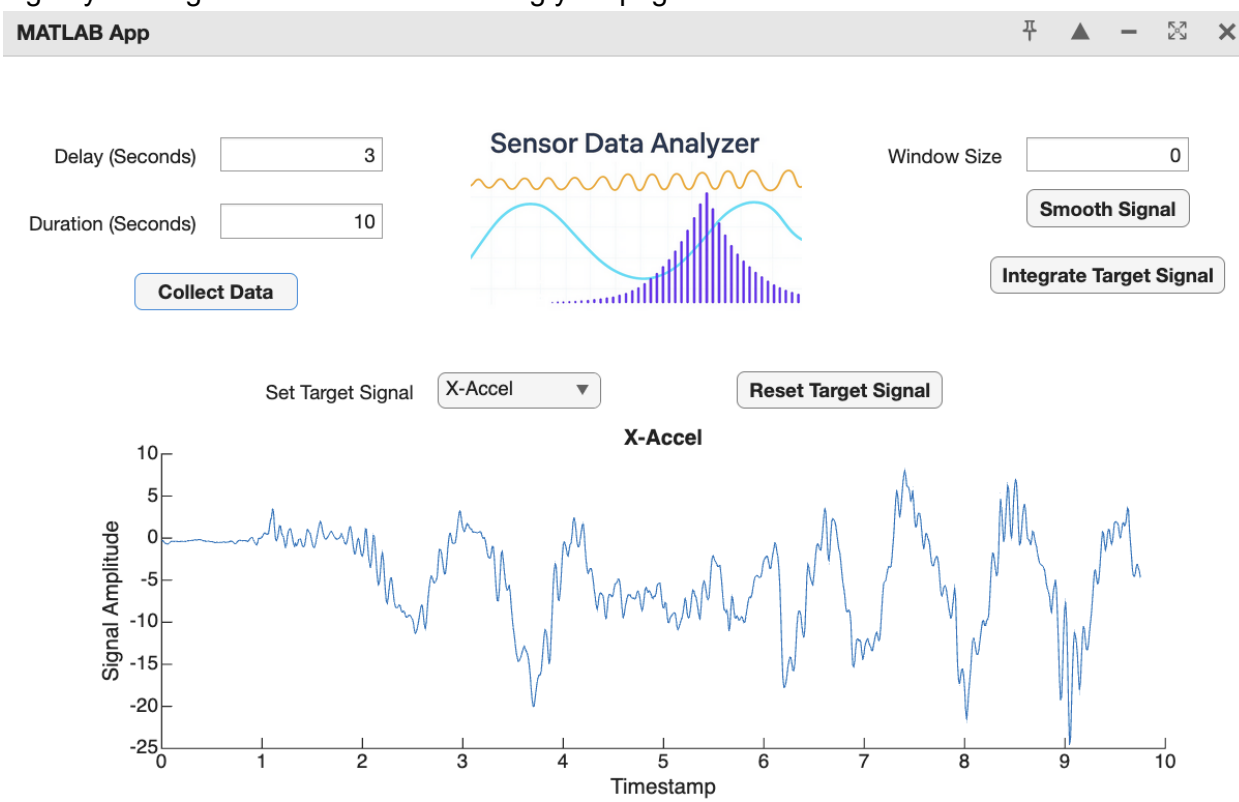


**Figure 8**: App with a logo image.

## What to Submit

Submit a PDF file according to the template provided. Also, submit your .mlapp file on Canvas. **Important**: your app **must** run without error. Even if you do not complete all the required functionality, make sure the functionality you do have works. Apps that do not run will receive very little partial credit.

Using your app, repeat the dead-reckoning experiment from step 3.2 in mini-project 4. In your lab report, submit screenshots of your app that include:

- The raw accelerometer measurements.
- Smoothed measurements of the above.
- An estimate of displacement calculated using numerical integration and dead reckoning.

# Rubric

- GUI is designed and implemented properly:
    - All required UI components are present and functional
    - The GUI is intuitive, good layout, not cluttered
    - The logo is visible, unique, and enhances the look
- Code is functional:
    - The target signal can be set/modified
    - Callback functions are implemented and connected
    - Each button performs its intended function
    - Plotting is implemented accurately and is
- App functionality is demonstrated successfully: Screenshots show
    - Raw sensor data
    - Smoothed signal
    - Integrated signal
    - Reset signa;
- Lab report is complete and well-formatted